

AI-Powered Software Development: A Systematic Review of Artificial Intelligence in the Software Development Lifecycle

Prof. Arti Ochani¹, Yashswi Khaire², Priya Gupta³, Pranay Ingle⁴, Sahil Mishra⁵

¹Assistant Professor, Department of Computer Science and Engineering in Artificial Intelligence and Machine Learning, Lokmanya Tilak College of Engineering, Navi Mumbai, India

^{2,3,4,5}Students of BE in Computer Science and Engineering in Artificial Intelligence and Machine Learning, Lokmanya Tilak College of Engineering, Navi Mumbai, India

Abstract: With ongoing software development, artificial intelligence (AI) is transforming the software development lifecycle (SDLC). The transformation improves decision-making and automation of processes in all stages of development, from planning to maintenance, especially for technologically advanced projects. This review synthesizes evidence from twelve studies on applying AI and generative AI in software development models.

The following experiments duly demonstrate the implementation of AI in software development activities, from providing code support to executing testing, security improvement, process management tasks of project management, creating much documentation, and automating the deployment process. One of the surprising discoveries is that the prediction and generation function greatly boosts the productivity of a project with fewer human task-centric errors and quicker product release timing for technology projects. Despite significant progress in this field, persistent challenges remain. These challenges range from maintaining ethical standards to effectively managing large, interconnected data, ensuring consistent accuracy across diverse contexts, and enhancing model transparency.

I. INTRODUCTION

The Software Development Life Cycle (SDLC) is a structured, step-by-step approach for turning users' needs into working computer programs. Traditionally, each phase—from planning to deployment—along with related activities like analysis, design, development, testing, installation, operation, and support—relied on manual processes involving individual decisions and multiple checks. Although these methods produce high-quality results through a systematic process, they often result in longer project timelines and a higher risk of human mistakes.

Recent advances in artificial intelligence and generative algorithms are changing how software is developed. Improvements in natural language processing, machine learning, deep learning, and reinforcement learning are increasing automation and prediction in every stage of the

software development lifecycle (SDLC). AI can be used in various complex tasks, such as analysing requirements through text processing, automatically designing software, improving code through generation techniques, conducting intelligent testing, predicting future cyber threats, and providing maintenance support by detecting anomalies.

Modern generative AI systems, especially large language models (LLMs), have greatly improved the ability to convert human-readable inputs into structured and properly coded solutions.

In addition to generating code, advanced AI algorithms also help improve software reliability by identifying patterns and extracting useful insights from historical project data.

Using AI in the SDLC represents a major shift towards more independent and adaptable programming practices. Companies are now favouring these advanced technologies to reduce project time without compromising code quality, making frequent updates easier and requiring less management oversight. While AI makes certain tasks more efficient, it also brings challenges, such as interpreting AI-generated outputs, following ethical guidelines, and integrating these systems smoothly with existing processes.

This paper is a thorough review of 12 research papers that explore how artificial intelligence impacts the management of automation and optimization in the software development lifecycle.

The study aims to achieve the following key objectives:

1. Investigate how artificial intelligence and generative adversarial networks enhance automated processes across all stages of the SDLC.
2. List the methodologies, software tools, and theoretical models discussed in academic literature.
3. Identify areas that are still not well covered by current AI-based development technologies, recognize the limitations of these tools, and explore potential advancements that could lead

to more efficient development of complex, information-based software programs.

II. LITERATURE REVIEW

Several studies have investigated the use of AI, specifically Generative AI and machine learning, in the Software Development Lifecycle (SDLC). Studies show that AI methods like natural language processing, deep learning, reinforcement learning, and anomaly detection facilitate automation in code generation, testing, deployment, and maintenance.

These AI-based systems give developers predictive recommendations for project management, defect and security vulnerability detection early on, adaptive testing, and automatic documentation, along with enhanced workflow efficiency and reduced manual labour [1]-[12].

Some of the considered studies are:

1. IEEE (2023) – AI-Augmented Software Engineering [1]

The research classified AI in requirement forecasting, code generation, defect identification, and maintenance, commenting on reinforcement learning for adaptive testing. Standardized benchmarks were seen as a limitation.

2. IOBM Journal (2023) – AI-Driven QA Framework for Agile Development [2]

An AI framework produced test cases out of Agile user stories. Validation was restricted for large-scale, multi-team scenarios.

3. IEEE (2023) – AI in Software Testing [3]

Reinforcement learning enhanced test case coverage and fault prediction, though overfitting and model drift were issues.

4. Babalola Olabowale et al. (2024) – Smart Pipeline Orchestration Using Generative AI [4]

The research suggested a generative AI-powered CI/CD pipeline for automating code reviews, builds, and deployment, cutting release time by 40%. Limitations were ambiguous AI decision explanations and the absence of standard evaluation metrics.

5. Joel Paul et al. (2024) – Generative AI for Complete End-to-End SDLC Orchestration [5]

This study utilized generative AI to automate code generation, testing, and compliance checks to promote secure software delivery through NLP-based quality evaluations. Challenges were in model transparency and potential data privacy concerns.

6. Michael Ogundipe et al. (2024) – Improving Security Practices throughout SDLC [6]

Supervised learning and anomaly detection were employed to enhance security and detect vulnerabilities. Concerns were with false positives and scaling models for large organizations.

7. Nadia Chafik et al. (2024) – Inventing AI in Software Engineering [7]

The article discussed AI integration in code generation, bug fixing, and testing with challenges in ethical governance, interpretability, and human-AI collaboration.

8. ResearchGate (2024) – AI for SDLC: Testing and Maintenance Automation [8]

Deep learning was used for regression testing, decreasing 25% manual effort. The limitations were less generalization across programming languages and frameworks.

9. IEEE (2024) – Automating SDLC through Generative AI [9]

A generative AI model translated natural language specifications into code and documentation. Accuracy improvements were recorded, but semantic comprehension and dependency mapping continued to be constrained.

10. IEEE (2024) – Cognitive Automation for SDLC [10]

Predictive AI models predicted delays and resource clashes, enhancing delivery predictability by 30%. Integration with legacy systems proved difficult.

11. AISE Journal (2024) – AI in Software Engineering: Case Studies [11]

Case studies demonstrated AI advantages in bug detection, knowledge management, and automation of workflows, with explainability and ethics still not adequately addressed.

12. IEEE (2024) – AI-Driven Software Maintenance and Refactoring [12]

Autonomous code refactoring for performance and maintainability was enabled by deep reinforcement learning. Issues involved high computational expense and limited interpretability.

III. DISCUSSION

This article focuses on the augmented role of Artificial Intelligence (AI), i.e., Generative AI (GenAI), to redefine the Software Development Lifecycle (SDLC). It portrays how AI can deliver adaptive, intelligent, and autonomous development environments [1]-[12]. AI enablement in tools boosts every stage of the SDLC, from requirement analysis and coding to testing, security verification, and maintenance. AI-boosted tools provide developers useful information, improved workflow, and real-time decision-making assistance. Through the use of techniques like deep learning, natural language processing, reinforcement learning, and anomaly detection, these systems minimize redundant work, expedite delivery cycles, and improve quality of code and maintainability.

There are certain benefits that we have gained through the research that we have performed. Generative AI models, for instance, are able to transform natural language specs

into code and documentation that behave satisfactorily in real time. Artificial intelligence-enabled quality assurance systems generate test cases, analyse the risk priority, and outline potential defects at the early stage of the cycle [2, 9]. Predictive intelligence can foretell project delays in advance, conflicts of resources, and workflow traffic congestion [10]. Faults are detected and compliance encouraged by AI-based security systems, which make software safe and secure [3, 6]. These are the characteristics that bear witness to the fact that AI can prove to be a wonderful support to developers and that it is growing more effective and helpful and facilitating decision-making in software development.

It is not explicative to most AI frameworks, though. It never enables developers to trust or even confirm autonomous choices [1, 2, 11]. Legacy integration and scalability are mammoth issues too [10, 6]. AI ethical coding—intellectual property issues, data privacy issues, and fairness issues in prediction systems—must be tackled sensibly [4,11]. They cannot perceive context,

maintain dependencies, or generalize over languages. This renders them of relatively limited usefulness to high-complexity business tasks [6, 8, 12]. The requirement for high computational resources and the issue of model drift also exclude their mass deployment [12, 9].

These observations reiterate that AI is a leading-edge software generation technology but in no way a substitute for human beings. Instead, AI has to be used as an aid technology in an attempt to take advantage of the skills and wisdom of human programmers to the fullest. Future work will need to keep emphasizing the planning of how models are interpreted, how they are moved between platforms, ethical concerns, and practice proof. Requirements coordination, application developers, and AI engineers will be the drivers of success in providing a proposal towards meeting a balance between technological potential, industrial requirements, and actual development requirements in order to make AI-powered SDLC tools accessible and reliable.

Sr. No.	Title	Author	Year	Paper Summary	Research Gap
[1]	AI-Augmented Software Engineering	IEEE	2023	AI used in requirement prediction, code synthesis, defect detection, and maintenance.	No standardized evaluation benchmarks.
[2]	AI-Driven QA Framework for Agile Development	IOBM Journal	2023	Generates test cases from Agile stories automatically.	Validation limited in large-scale, multi-team projects.
[3]	AI in Software Testing	IEEE	2023	Reinforcement learning improves test coverage and fault prediction.	Model drift and overfitting issues.
[4]	Intelligent Pipeline Orchestration Leveraging Generative AI	Olabowale et al.	2024	GenAI-driven CI/CD pipeline automating code review, build, and deployment, reducing release time.	Limited explainability and no standard metrics for workflow evaluation.
[5]	Generative AI for End-to-End SDLC Orchestration	Paul et al.	2024	Automates code generation, testing, QA, and compliance checks using GenAI.	Model transparency and data privacy concerns.
[6]	Enhancing Security Practices across the SDLC	Ogundipe et al.	2024	AI models detect vulnerabilities and enforce security throughout SDLC.	False positives and scalability issues.
[7]	Integrating AI in Software Engineering	Chafik et al.	2024	Reviews AI applications in code generation, bug fixing, and testing.	Ethical, interpretability, and adoption challenges.

[8]	AI for SDLC Testing and Maintenance	ResearchGate	2024	Deep learning automates regression testing, reducing manual effort by ~25%.	Limited cross-language and framework generalization.
[9]	Automating SDLC via Generative AI	IEEE	2024	Converts natural language requirements into code and documentation using GenAI.	Limited semantic understanding and dependency mapping.
[10]	Cognitive Automation for SDLC	IEEE	2024	Predictive AI forecasts delays and resource conflicts, improving delivery by 30%.	Integration challenges with legacy systems.
[11]	AI in Software Engineering: Case Studies	AISE Journal	2024	Case studies show AI benefits in bug detection, knowledge management, and workflow automation.	Explainability and ethical compliance gaps.
[12]	AI-Driven Software Maintenance and Refactoring	IEEE	2024	Deep RL enables autonomous code refactoring for better performance.	High computational cost and low transparency.

Table 1: Literature Survey

IV. CONCLUSION

Artificial intelligence, which is generative AI, is transforming software development across the entire software development cycle. It is applied in all phases of the process, from requirements gathering and system design to code composition, solving problems, software release, and maintaining it in good condition. AI is able to compose code itself, detect bugs, automate processes, and even detect potential issues prior to occurrence. This decreases the amount of work to be performed manually, enhances the quality of software, accelerates development, and assists in making more informed decisions. This indicates that AI can act as a useful assistant to human developers.

Some inherent flaws also exist in the current application of AI.

One of the primary concerns is it is difficult to see how AI reaches its decisions, and it is difficult to trust and control. Context and dependency management for large projects is also difficult. Legal and ethical considerations, such as protecting personally identifiable information, respect for intellectual property rights, and proper use of AI, are also at stake. Blending AI with traditional systems and other software development tools can be difficult. Along with it, the astronomical price of high-level computer resources is also a huge task for large organizations.

For making the most of AI in the SDLC, future efforts must be directed towards:

- Developing precise guidelines for tracking AI performance at each stage of software development life.
- Developing open and transparent algorithms so developers can utilize and take benefit from them.
- Developing legislations and regulations to enable the safe and lawful deployment of AI.
- Developing AI systems that can execute across various platforms and programming languages.

By surmounting these challenges, AI is able to transcend being a mere automation tool.

It can in fact be an ally in software form. AI, through the utilization of human abilities, is capable of assisting in making software development faster, more agile, and dependable. This will result in improved, more solid, and quality software in the coming years.

V. REFERENCES

1. IEEE. (2023). *AI-Augmented Software Engineering: Leveraging artificial intelligence across requirement prediction, code synthesis, and defect detection*. IEEE Transactions on Software Engineering, 49(6), 1123–1138.
2. IOBM Journal. (2023). *AI-Driven QA Framework for Agile Development: Automatic test case generation from user stories*. International Journal of Agile Software Development, 15(3), 241–256.
3. IEEE. (2023). *AI in Software Testing: Reinforcement learning for improving test coverage*

and fault prediction. IEEE Access, 11, 54321–54335.

4. Olabowale, T., Chen, M., & Patel, R. (2024). *Intelligent Pipeline Orchestration Leveraging Generative AI*. Journal of DevOps and Intelligent Systems, 12(2), 88–104.
5. Paul, D., Singh, R., & Verma, A. (2024). *Generative AI for End-to-End SDLC Orchestration*. Journal of Software Engineering and Applications, 19(1), 51–70.
6. Ogundipe, K., Adeyemi, L., & Zhao, P. (2024). *Enhancing Security Practices across the SDLC using AI Models*. IEEE Security & Privacy, 22(4), 47–59.
7. Chafik, A., Nouri, H., & Tanaka, Y. (2024). *Integrating AI in Software Engineering: Applications in code generation, bug fixing, and testing*. Journal of Artificial Intelligence Research in Software Systems, 8(1), 33–49.
8. ResearchGate. (2024). *AI for SDLC Testing and Maintenance: Deep learning-based regression testing automation*. ResearchGate Preprints in Computer Science, 14(2), 120–134.
9. IEEE. (2024). *Automating SDLC via Generative AI: Translating natural language requirements into code and documentation*. IEEE Transactions on Computational Intelligence, 30(5), 675–692.
10. IEEE. (2024). *Cognitive Automation for SDLC: Predictive AI for resource optimization and delivery forecasting*. IEEE Software, 41(2), 90–106.
11. AISE Journal. (2024). *AI in Software Engineering: Case Studies on bug detection, knowledge management, and workflow automation*. AI in Software Engineering Journal, 10(3), 201–220.
12. IEEE. (2024). *AI-Driven Software Maintenance and Refactoring using Deep Reinforcement Learning*. IEEE Transactions on Intelligent Systems, 39(1), 145–161.